Heatwave Prediction Final Project Report

For this project I used the Rajasthan Heatwave dataset from Kaggle, building on the preprocessing I already did for my midterm. The goal is binary classification, predicting whether a day/location combo sees a heatwave (HEATWAVE = 1 or 0), using weather features like wind, pressure, boundary layer height, geopotential, soil conditions, and district. The dataset's heavily imbalanced, about 95% no-heatwave and 5% heatwave. I handled that part in the midterm with SMOTE on the training data only, after splitting, so validation and test still reflect real-world proportions.

From my midterm I have a 70/30 train/test split with missing values filled from training stats, one-hot encoded, numeric features scaled with StandardScaler fit on training only, and SMOTE balancing the training set. For this final project I used a 70/15/15, rather than reprocessing everything again, I just split my existing 30% test set in half to get a 15% validation set and 15% test set. Both kept the real imbalance, since only training data ever got SMOTE applied to it.

My first full run results had leakage since, Decision Tree, Random Forest, Gradient Boosting, and both ensembles all hit a perfect 100% on every metric, on both validation and test. From our lessons I know that a 100% across on multiple models is rare if possible. I checked Random Forest's feature and found TMAX, TMIN, and TEMP2M made up about 79% of what it was using. This was because heatwaves are basically defined by temperature crossing a limit/cutoff, so the model wasn't predicting, it was just reading the label off those columns. I dropped those three and retrained everything. Here is the result below which reflects a more honest test of whether indirect signals like wind, pressure, humidity, soil, and location can actually predict a heatwave without cheating off the label.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.936	0.433	0.945	0.594	0.977
Decision Tree	0.951	0.505	0.677	0.578	0.821
Random Forest	0.966	0.632	0.774	0.696	0.980
Gradient Boosting	0.936	0.430	0.878	0.577	0.980
KNN	0.920	0.368	0.841	0.512	0.936
SVC	0.951	0.505	0.927	0.654	0.986
Voting Ensemble	0.961	0.573	0.884	0.695	0.985
Bayesian Ensemble	0.961	0.574	0.878	0.694	0.985

Random Forest gave me the strongest standalone results, best F1 0.696 and best precision 0.632 on test. I think that's because it combines a bunch of trees trained on different feature subsets, so it can pick up on interactions like wind direction with pressure and soil moisture without overfitting to one threshold the way a single Decision Tree does. The Voting and Bayesian ensembles landed almost the same as each other and slightly ahead on ROC-AUC 0.985 and recall +0.88, but with lower precision 0.57 than Random Forest alone. This part was explained to me with the help of Gemini, “Averaging in SVC and Gradient Boosting pushed the ensemble toward flagging more positives, more true catches but more false alarms too. Weighting the Bayesian ensemble by F1-score barely changed anything since the weights came out close to equal anyway”. KNN was my weakest model by a lot, F1 just 0.512 and precision 0.37. It relies on raw distance between points, which struggles once SMOTE packs in synthetic neighbors, so it over-predicts the minority class and racks up false positives.

I would lean toward the ensembles for better recall and ROC-AUC. But since false alarms are the bigger cost, Random Forest's precision makes more sense. I'd recommend Random Forest as the baseline since it balances both reasonably well.